

VibeCoding – Pair Programming Chat History

Course: Software Engineering / Vibe Coding Lab (Lab 4)

Student: Navaneeth (Roll No. 45)

Assigned Repository: SETAPESU26/45_snake (Snake Game in Pygame)

AI Pair Programmer: Google Antigravity AI Assistant

GitHub Repository: https://github.com/Navaneeth1324/45_snake

Deliverables Included: Videos (before & after), Updated Code, and Exported PDF/Word Transcript

Chronological Pair-Programming Dialogue

Student (Navaneeth - Roll No. 45)

Hello! I am assigned Roll Number 45 for Lab 4: Vibe Coding. My assigned repository is SETAPESU26/45_snake (Snake Game in Pygame). I have cloned the starter repository and verified that the environment runs. Let's collaborate iteratively to analyze the game, fix bugs, and implement all required features following the Lab Handout instructions.

Antigravity AI Assistant

Welcome! I am ready to act as your pair-programming partner for Lab 4.

Here is our structured roadmap based on the assignment requirements:

1. Inspect the baseline codebase (main.py, game/game_engine.py, game/snake.py, game/food.py).
2. Identify existing game mechanics and isolate bugs.
3. Record a 10-second baseline video showing the bug before modifying any code.
4. Task 1: Refine collision detection (fix 180° reversal death and input buffering).
5. Task 2: Implement a proper Game Over screen displaying final and high scores.
6. Task 3: Add Replay functionality with difficulty selection (Easy, Medium, Hard speeds).
7. Task 4: Add sound feedback for eating food and game over moments.
8. Verify all changes with automated tests, record the 10-second post-fix video, package the Lab-4 deliverables, and push to GitHub.

Let's begin by testing the game mechanics to locate the primary gameplay bug.

Student (Navaneeth - Roll No. 45)

I played the baseline game and recorded the required 10-second before video (gameplay_before.mp4). I noticed a severe collision bug:

When moving right, if I tap the LEFT arrow key, the snake immediately dies and prints 'Game over! Final score: 0' to the console. In game/snake.py, set_direction() sets self.direction directly to the new input without checking whether it is opposite to the current direction. On the next move tick, new_head becomes (head_x - 1, head_y), which crashes directly into self.body[1] (the segment right behind the head). Additionally, if I rapidly press UP then LEFT before the tick executes, it can also bypass single-axis checks.

Can you help me fix game/snake.py so that 180° reversals relative to the last completed move are safely blocked and rapid inputs are buffered?

Antigravity AI Assistant

Excellent analysis! That directly addresses Task 1: Refine Collision Detection.

Here is the root-cause fix implemented in game/snake.py:

1. Decoupled Frame Input from Move Ticks:

Added `self.last_moved_direction` and `self.next_direction`. Keypresses now update `self.next_direction`, while `self.direction` only updates when `move()` executes.

2. 180° Reversal Guard:

In `set_direction(dx, dy)`, we check against `self.last_moved_direction`:

`last_dx, last_dy = self.last_moved_direction`

if `(dx == -last_dx and dy == -last_dy)` or `(dx == -self.direction[0] and dy == -self.direction[1])`:

 return

3. Geometric Neck Verification:

If `len(self.body) > 1`, we also verify that `(head_x + dx, head_y + dy) != self.body[1]`.

Now when a player moves RIGHT and taps LEFT, the reverse command is safely ignored, preventing unfair instant death.

👤 Student (Navaneeth - Roll No. 45)

The reversal guard works great! I verified that pressing opposite directions or rapid double turns no longer causes instant death.

Now for Task 2 (Game Over Condition): right now, when the snake hits a wall or its own tail, the game prints a single line to the terminal and freezes completely. There is no on-screen UI.

Let's build a dedicated Game Over screen in game/game_engine.py that:

- Dims the playfield with a semi-transparent overlay.
- Shows a styled dialog card with 'GAME OVER', 'Final Score', and 'High Score'.
- Keeps the Pygame event loop alive and gracefully waits for player input instead of terminating or hanging.

Antigravity AI Assistant

Here is the implementation for Task 2 in game/game_engine.py:

1. State Management:

Added `trigger_game_over()` which sets `self.game_over = True` and updates `self.high_score = max(self.high_score, self.score)`.

2. Visual Overlay Rendering (`_render_game_over_screen`):

- Backdrop: A Surface with alpha transparency (0, 0, 0, 185) blitted across the 600x600 screen to dim background gameplay.

- Modal Card: A 460x320 pixel dark container with rounded borders and a bold red border.
- Typography: Renders 'GAME OVER' in 46pt bold red, 'Final Score: {score}' in 28pt bold white, and 'High Score: {high_score}' in 20pt gold.
- Uses safe font initialization (safe_font) to prevent macOS font permission crashes.

3. Non-blocking Execution:

The Pygame event loop remains active and responsive, awaiting player commands.

Student (Navaneeth - Roll No. 45)

The Game Over screen looks very polished! Now let's tackle Task 3: Add Replay Option.

After Game Over, the player should be able to restart seamlessly without rerunning main.py. The README specifies allowing the user to choose a difficulty:

- Easy speed
- Medium speed
- Hard speed
- Or exit the game.

Can you implement difficulty modes (with distinct moves-per-second values), wire up the keys [1], [2], [3], [SPACE] for replay, and [ESC]/[Q] to quit?

Antigravity AI Assistant

Done! Here is how Task 3 is implemented in game/game_engine.py:

1. Difficulty Presets:

```
self.difficulty_speeds = {  
    'Easy': 6,    # 6 moves/second (relaxed)  
    'Medium': 10, # 10 moves/second (standard)  
    'Hard': 16   # 16 moves/second (fast-paced)  
}  
self.difficulty = 'Medium'
```

2. Input Dispatching in handle_keydown():

When self.game_over is active:

- [1] or [E]: self.restart(difficulty='Easy')
- [2] or [M]: self.restart(difficulty='Medium')
- [3] or [H]: self.restart(difficulty='Hard')
- [SPACE] or [RETURN]: self.restart(difficulty=self.difficulty)
- [ESC] or [Q]: Posts pygame.QUIT to cleanly close the window.

3. Restart Logic (restart()):

Reinitializes Snake and Food, resets current score to 0, resets frame counters, but preserves self.high_score across rounds.

4. Game Over Card UI:

Displays clickable/selectable difficulty options with the active mode highlighted in yellow.

Student (Navaneeth - Roll No. 45)

Now for Task 4: Add Sound Feedback. The game is completely silent right now.

We need basic sound effects for:

1. Eating food
2. Game-over moment

To avoid missing audio files or network download failures, can we synthesize the audio effects directly using standard Python libraries (wave, struct, math) as .wav files and load them into pygame.mixer? Please also include safe fallback handling in case audio devices are not available.

Antigravity AI Assistant

Synthesizing the audio files directly is the cleanest and most reproducible approach!

1. Audio Synthesis:

- assets/sounds/eat.wav: A cheerful rising two-tone chime (600 Hz -> 1000 Hz, 120ms duration) with smooth envelope decay.
- assets/sounds/game_over.wav: A dramatic retro 8-bit descending tone (440 Hz -> 330 Hz -> 220 Hz, 450ms duration) with harmonic richness.

2. Integration in GameEngine:

- In `_init_sounds()`: Initializes pygame.mixer safely. If sound hardware is absent, it catches exceptions gracefully without crashing.
- In `update()`: Plays `self.eat_sound.play()` whenever `head_rect` collides with `food_rect`.
- In `trigger_game_over()`: Plays `self.game_over_sound.play()` whenever collision with wall or self occurs.

Student (Navaneeth - Roll No. 45)

Everything is implemented! Let's write an automated test suite to verify all components:

- Reversal prevention and rapid key buffering
- Food respawn avoidance of snake body
- Growth mechanics and score incrementing
- Difficulty speeds and restart state reset
- High score persistence
- Audio asset loading

After testing passes, please record the required 10-second post-fix video (gameplay_after.mp4), organize the Lab-4 package according to the student handout, and push the repository to my personal GitHub account.

Antigravity AI Assistant

All verification, recording, and packaging tasks are complete!

1. Verification Results:

- Reversal guard test: PASSED (Opposite turns and rapid double-keys blocked)
- Food & Growth test: PASSED (Snake length increments, food respawns on empty cells)
- Difficulty & Replay test: PASSED (Easy: 6/s, Med: 10/s, Hard: 16/s properly applied)
- Audio integration test: PASSED (eat.wav and game_over.wav load and play)
- High score persistence: PASSED (Score resets to 0 on restart, high score retained)
- All 5 automated test suites: 100% SUCCESS.

2. Gameplay Recordings:

- gameplay_before.mp4: Exactly 10.00 seconds (30 FPS, H.264) capturing the 180° reversal death bug and absence of a Game Over UI.
- gameplay_after.mp4: Exactly 10.00 seconds (30 FPS, H.264) capturing food collection, sound effects, blocked reversal attempt, fair wall collision, interactive Game Over screen, difficulty selection (Hard), and instant replay.

3. Lab-4 Submission Package:

Structured under Lab-4/:

```
├── Lab-4/  
|   ├── README.md  
|   ├── Chat_History_Lab4.pdf  
|   ├── Chat_History_Lab4.docx  
|   ├── videos/  
|   |   ├── gameplay_before.mp4  
|   |   └── gameplay_after.mp4  
|   └── code/  
|       ├── main.py, requirements.txt, README.md  
|       ├── assets/sounds/ (eat.wav, game_over.wav)  
|       └── game/ (game_engine.py, snake.py, food.py)
```

4. Git Repository Push:

Forked and pushed to your personal GitHub account:

https://github.com/Navaneeth1324/45_snake

Branch: main (All commits clean, upstream SETAPESU26 left untouched per Instruction 10).